LING 510: Week 10 review*

November 13, 2026

Shaunak Phadnis
sphadnis@umass.edu
Office hours: W 1:30–3:30PM, N421H | Zoom

1 Free and bound pronouns

We have dealt with pronouns whose interpretation is context (or assignment dependent) – they were **free**. But a pronoun's interpretation can also be bound under certain linguistic environments i.e., meaning of a pronoun **co-varies** with another DP in the structure.

(1) a. Every woman likes her job.
⇝ Abigail likes Abigail's job, Aditi likes Aditi's job, Stevie likes Stevie's job …
b. Only Shawn likes his cat.
⇝ Barry doesn't like Barry's job, Smith doesn't like Smith's job …

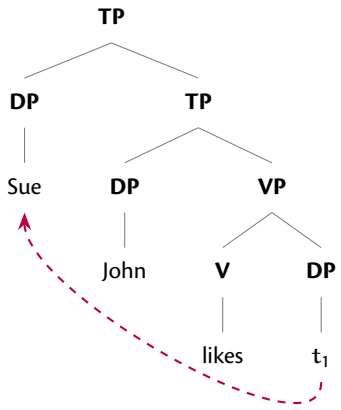
Pronouns that get a bound reading can, in principle, get a free reading as well in the same syntactic context. This is most transparent in cases of gender mismatch.

(2) Every man likes her job. (*Let's say, Abigail's*)

2 Movement, traces, and relative clauses

When a constituent moves, it is pronounced in a place that is not the place where it gets interpreted. For instance, consider topicalization in (3).

(3) Sue, John likes t_1.



The DP Sue is pronounced at the front of the sentence, but it is interpreted as the object of likes. We don't want the interpretation to be *free*! Let's say we had the following assignment.

*Material in this handout is based on Andrea Matticchio's Spring '26 lab handouts for LING510.

$$g = \begin{bmatrix} 1 \mapsto \text{Stella} \\ 2 \mapsto \text{Billy} \\ 3 \mapsto \text{John} \\ \vdots \end{bmatrix}$$

Derivation without predicate abstraction. Sue, John likes t_1, relative to the original g (so $g(1) =$ Stella). Treat the trace like a pronoun.

1. $[\![t_1]\!]^g = g(1) = \text{Stella}$ (PR)
2. $[\![\text{likes } t_1]\!]^g = [\lambda x_e.\, [\lambda y_e.\, \text{T iff y likes x}]](\text{Stella}) = [\lambda y_e.\, \text{T iff y likes Stella}]$ (FA, TN, 1, β)
3. $[\![\text{John likes } t_1]\!]^g = [\lambda y_e.\, \text{T iff y likes Stella}](\text{John}) = \text{T iff John likes Stella}$ (FA, TN, 2, β)
4. $[\![\text{Sue, John likes } t_1]\!]^g = ??$ The daughters are $[\![\text{Sue}]\!]^g = \text{Sue}$ (type e) and $[\![\text{John likes } t_1]\!]^g$ (type t). FA needs one daughter to be a function, and neither is. PM needs two $\langle e, t\rangle$s. **No rule applies.**

What went wrong

1. **Type mismatch.** The moved DP (type e) and the lower TP (type t) cannot combine, so the sentence gets no denotation at all.
2. **Wrong value for the trace.** Even if we ignored Sue, the trace took its value from the context: we computed *John likes Stella*. We want *T iff John likes Sue*.

Both problems have one source: nothing in the tree connects Sue to the index on t_1.

What we need is a mechanism that will keep rest of the assignments intact but only changes the assignment for the index appearing in the tree.

$$g = \begin{bmatrix} 1 \mapsto \text{Stella} \\ 2 \mapsto \text{Billy} \\ 3 \mapsto \text{John} \\ \vdots \end{bmatrix} \quad \Longrightarrow \quad \underbrace{g = \begin{bmatrix} 1 \mapsto \text{Sue} \\ 2 \mapsto \text{Billy} \\ 3 \mapsto \text{John} \\ \vdots \end{bmatrix}}_{\text{assignment we want}}$$

Note that the source of our problem lies in the syntax – **not** in how the assignment was defined.

- We want to overwrite the assignment $g(1) = \text{Stella}$ to $g(1) = \text{Sue}$.
- We want the moved constituent Sue to be used up to do so.
- We want rest of the things to remain the same.

In a way, we need g to temporarily map 1 to a variable that can then be mapped to Sue in the course of the derivation!

Definition: Modified assignment function

Given any assignment function g, the modified assignment function $g[n \mapsto a]$[a] is the assignment function just like g except—at most—that it maps n to a.

$$g[n \mapsto a](m) = \begin{cases} a & \text{if } m = n \\ g(m) & \text{if } m \neq n \end{cases}$$

Alternative notation: $g[n/a](m)$

[a] Do not be confused by the notation. A function's argument is always enclosed in round braces '( )' and never in square braces '[ ]'.

In a modified assignment function g, all assignments are the same as before *except* the one which is modified.

$$g = \begin{bmatrix} 1 \mapsto a \\ 2 \mapsto b \\ 3 \mapsto c \\ \vdots \\ m \mapsto u \\ \vdots \end{bmatrix} \quad \Longrightarrow \quad g[m \mapsto a] = \underbrace{\begin{bmatrix} 1 \mapsto a \\ 2 \mapsto b \\ 3 \mapsto c \\ \vdots \\ m \mapsto a \\ \vdots \end{bmatrix}}_{\text{modified assignment}}$$

Per the original assignment $g(m) = u$ but according to the modified assignment $g(m) = a$.

Since, Sue is ultimately going to be used up via function application, it needs to be an argument of some function that inputs entities!

Predicate Abstraction Rule (PA)

If α is a branching node whose daughters are β_i and γ, where β_i is a binder index i (left behind by movement) or a relative pronoun bearing the index i, then for any assignment g,

$$\left[\!\!\left[\begin{array}{ccc} & \alpha & \\ \beta_i & & \gamma \end{array} \right]\!\!\right]^g = \lambda x_e.\, [\![\gamma]\!]^{g[i \mapsto x]}$$

where for each object x, $g[i \mapsto x]$ is the modified assignment defined by:

- for each index $j \neq i$, $g[i \mapsto x](j) = g(j)$;
- and $g[i \mapsto x](i) = x$.

1. When a DP moves, it leaves behind a **binder index** $\underline{i}$ right below its landing site, and a trace t_i in its original position.

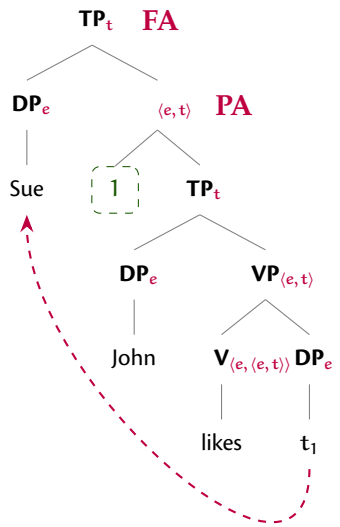
2. PA applies at the node where the binder index is a daughter. It builds a function $\lambda x_e.\, [\![\gamma]\!]^{g[i \mapsto x]}$, in which the trace t_i inside γ picks out whatever x is, no matter what g said about i.

3. The moved DP then combines with that function by FA, so it gets used up.

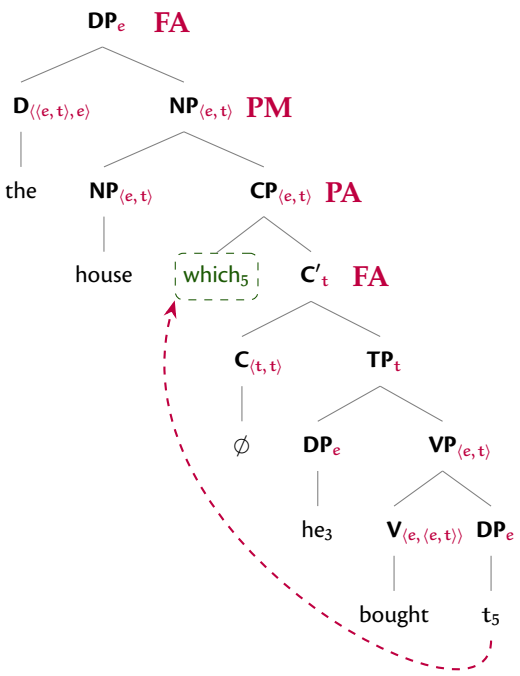
4. The same rule will handle relative clauses, where the relative pronoun who_i/which_i plays the role of the binder.

Here is our topicalized sentence (a), next to a relative clause (b) for comparison. Notice that they are structurally similar, in terms of composition, we'd have run into the same type mismatch at the CP node where FA could not have been applicable originally.

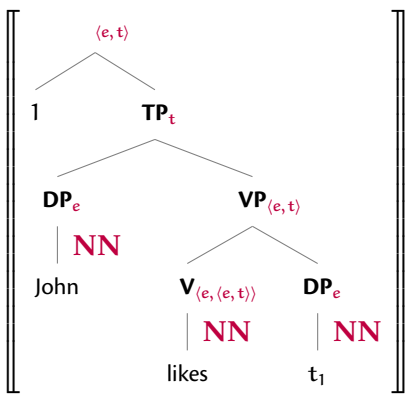
(a) Sue, John likes t_1



(b) the house which_5 he_3 bought t_5



Subproof. the sister of Sue, relative to g (where $g(1) = \text{Stella}$)

1. $\left[\!\!\left[\right.\right.$  $\left.\right]\!\!\right]^g = \lambda x_e.\, [\![\text{TP}]\!]^{g[1 \mapsto x]}$ (by PA)

2. $= \lambda x_e.\, [\![\text{VP}]\!]^{g[1 \mapsto x]}([\![\text{John}]\!]^{g[1 \mapsto x]})$ (by FA, NN)

3. $= \lambda x_e.\, [\![\text{likes}]\!]^{g[1 \mapsto x]}([\![t_1]\!]^{g[1 \mapsto x]})(\text{John})$ (by FA, NN, TN)

4. $= \lambda x_e.\, [\![\text{likes}]\!]^{g[1\mapsto x]}\big(g[1 \mapsto x](1)\big)(\text{John})$ (by PR)

> Note that the modified assignment $g[1 \mapsto x]$ looks like:
>
> $$g[1 \mapsto x] = \begin{bmatrix} 1 \mapsto x \\ 2 \mapsto \text{Billy} \\ 3 \mapsto \text{John} \\ \vdots \end{bmatrix}$$

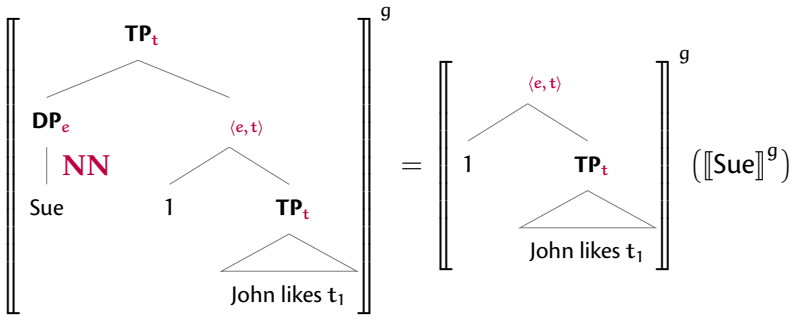
5. $= \lambda x_e.\, [\![\text{likes}]\!]^{g[1\mapsto x]}(x)(\text{John})$ (by def. of $g[1 \mapsto x]$)

6. $= \lambda x_e.\, [\lambda z_e.\, [\lambda y_e.\, \text{T iff } y \text{ likes } z]](x)(\text{John})$ (by TN)

7. $= \lambda x_e.\, \text{T iff John likes } x$ (by β-reduction 2×)

PA is where the binding happens: it introduces λx *and* shifts the assignment to $g[1 \mapsto x]$. When the trace t_1 is interpreted, it looks up index 1 and finds x, the very variable that λx binds (dashed arrows).

Derivation. higher TP, relative to g

1. 

(by FA)

2. $= [\lambda x_e.\, \text{T iff John likes } x]\big([\![\text{Sue}]\!]^g\big)$ (by Subproof)

3. $= [\lambda x_e.\, \text{T iff John likes } x](\text{Sue})$ (by NN, TN)

4. $= \text{T iff John likes Sue}$ (by β-reduction)

Practice

Can you derive the meaning of the relative clause the house which$_5$ he$_3$ bought?

We compute tree (b) relative to the same g as before, so $g(3) = \text{John}$.

Lexical entries (the same under every assignment g)

$$[\![\text{bought}]\!]^g = [\lambda x_e.\,[\lambda y_e.\,\text{T iff y bought x}]]$$
$$[\![\emptyset]\!]^g = [\lambda p_t.\,p]$$
$$[\![\text{house}]\!]^g = [\lambda x_e.\,\text{T iff x is a house}]$$
$$[\![\text{the}]\!]^g = [\lambda f_{\langle e,t\rangle} : \text{there is exactly one x in c such that } f(x) = T.\ \text{the unique x in c such that } f(x) = T]$$

Subproof a. CP, relative to g

1. $\left[\!\!\left[\begin{array}{c} \mathbf{CP}_{\langle e,t\rangle} \\ \text{which}_5 \quad \mathbf{C'}_t \\ \mathbf{C}_{\langle t,t\rangle} \quad\quad \mathbf{TP}_t \\ \mathbf{NN} \quad\quad\quad \mathbf{DP}_e \quad\quad \mathbf{VP}_{\langle e,t\rangle} \\ \emptyset \quad\quad\quad\ \mathbf{NN} \quad\quad\quad\ \mathbf{V}_{\langle e,\langle e,t\rangle\rangle} \quad \mathbf{DP}_e \\ \text{he}_3 \quad\quad\quad\quad\ \mathbf{NN} \quad\quad\quad\ \mathbf{NN} \\ \text{bought} \quad\quad\quad\quad\ t_5 \end{array}\right]\!\!\right]^g = \lambda x_e.\,[\![C']\!]^{g[5\mapsto x]}$ (by PA)

2. $= \lambda x_e.\,[\![\emptyset]\!]^{g[5\mapsto x]}\big([\![TP]\!]^{g[5\mapsto x]}\big)$ (by FA, NN)

3. $= \lambda x_e.\,[\![\emptyset]\!]^{g[5\mapsto x]}\big([\![VP]\!]^{g[5\mapsto x]}\big([\![\text{he}_3]\!]^{g[5\mapsto x]}\big)\big)$ (by FA, NN)

4. $= \lambda x_e.\,[\![\emptyset]\!]^{g[5\mapsto x]}\big([\![\text{bought}]\!]^{g[5\mapsto x]}\big([\![t_5]\!]^{g[5\mapsto x]}\big)\big([\![\text{he}_3]\!]^{g[5\mapsto x]}\big)\big)$ (by FA, NN)

5. $= \lambda x_e.\,[\![\emptyset]\!]^{g[5\mapsto x]}\big([\![\text{bought}]\!]^{g[5\mapsto x]}\big(g[5\mapsto x](5)\big)\big(g[5\mapsto x](3)\big)\big)$ (by PR 2×)

> Note that the modified assignment $g[5 \mapsto x]$ looks like:
>
> $$g[1 \mapsto x] = \begin{bmatrix} 1 \mapsto \text{Sue} \\ 2 \mapsto \text{Billy} \\ 3 \mapsto \text{John} \\ \vdots \\ 5 \mapsto x \end{bmatrix}$$
>
> The index 3 is left unmodified by this assignment, only 5 is assigned to a different value.

6. $= \lambda x_e.\,[\![\emptyset]\!]^{g[5\mapsto x]}\big([\![\text{bought}]\!]^{g[5\mapsto x]}(x)\big(g(3)\big)\big)$ (by def. of $g[5 \mapsto x]$)

7. $= \lambda x_e.\,[\lambda p_t.\,p]\big([\lambda z_e.\,[\lambda y_e.\,\text{T iff y bought z}]](x)\big(g(3)\big)\big)$ (by TN)

8. $= \lambda x_e.\,[\lambda p_t.\,p]\big(\text{T iff } g(3) \text{ bought } x\big)$ (by β-reduction 2×)

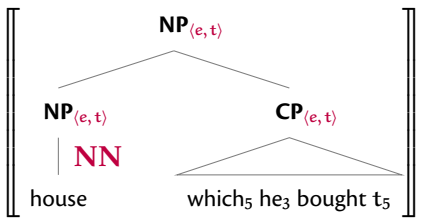
9. $= \lambda x_e.\,\text{T iff } g(3) \text{ bought } x$ (by β-reduction)

10. $= \lambda x_e.\,\text{T iff John bought } x$ ($g(3) = \text{John}$)

The relative pronoun which_5 shifts the assignment for index 5 only. So the trace t_5 is **bound**: it finds x, and whatever g says about 5 plays no role. The pronoun he_3 stays **free**: $3 \neq 5$, so $g[5 \mapsto x](3) = g(3)$, and its value still comes from the context.

At the higher NP both daughters, NP and CP, are of type $\langle e, t\rangle$. FA can't apply, so we use **PM**.

Subproof b. higher NP, relative to g

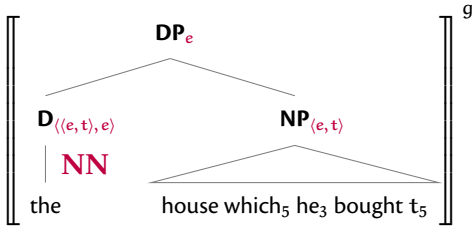
1. $\Big[\!\Big[$  $\Big]\!\Big]^g$

$$= \Big[\lambda x_e.\,\text{T iff } \Big[\!\Big[\text{NP}_{\langle e,t\rangle} \;\big|\; \text{NN}\ \text{house} \Big]\!\Big]^g(x) = \text{T and } \Big[\!\Big[\text{CP}_{\langle e,t\rangle}\ \text{which}_5\ \text{he}_3\ \text{bought}\ t_5 \Big]\!\Big]^g(x) = \text{T}\Big] \quad \text{(by PM)}$$

2. $= [\lambda x_e.\,\text{T iff } [\lambda z_e.\,\text{T iff } z \text{ is a house}](x) = \text{T}$
and $[\lambda z_e.\,\text{T iff John bought } z](x) = \text{T}]$ (by NN, TN, a.)

3. $= [\lambda x_e.\,\text{T iff } x \text{ is a house and John bought } x]$ (by β-reduction 2×)

Derivation. DP, relative to g

1. $\Big[\!\Big[$  $\Big]\!\Big]^g$

$$= \Big[\!\Big[\text{D}_{\langle\langle e,t\rangle,e\rangle} \;\big|\; \text{NN}\ \text{the} \Big]\!\Big]^g \Big(\Big[\!\Big[\text{NP}_{\langle e,t\rangle}\ \text{house which}_5\ \text{he}_3\ \text{bought}\ t_5 \Big]\!\Big]^g \Big) \quad \text{(by FA)}$$

2. $= [\![\text{the}]\!]^g\big([\lambda x_e.\,\text{T iff } x \text{ is a house and John bought } x]\big)$ (by NN, b.)

3. $= [\lambda f_{\langle e,t\rangle} : \text{there is exactly one } x \text{ in } c \text{ such that } f(x) = \text{T}.$
the unique x in c such that $f(x) = \text{T}]$
$\big([\lambda z_e.\,\text{T iff } z \text{ is a house and John bought } z]\big)$ (by TN)

4. **Presupposition check (the):** is there exactly one x in C such that x is a house and John bought x? Only if the answer is yes can we β-reduce. Otherwise the DP is **undefined**.

5. $=$ the unique x in c such that x is a house and John bought x (by β-reduction $2\times$)

The DP is of type e: it picks out the one house John bought. The buyer is John only because $g(3) =$ John. Under an assignment that maps 3 to someone else, the same DP picks out the house *that* person bought. The value of the trace never depends on the assignment.

Rules of interpretation, relative to an assignment g

Terminal Nodes (TN). If α is a terminal node that is not a pronoun or a trace, then, for any assignment g, $[\![\alpha]\!]^g$ is given by the lexicon.

Non-Branching Nodes (NN). If α is a node with sole daughter β, then, for any assignment g, $[\![\alpha]\!]^g = [\![\beta]\!]^g$.

Function Application (FA). If α is a branching node with daughters β and γ, and $[\![\beta]\!]^g$ is a function whose domain contains $[\![\gamma]\!]^g$, then, for any assignment g, $[\![\alpha]\!]^g = [\![\beta]\!]^g([\![\gamma]\!]^g)$.

Predicate Modification (PM). If α is a branching node with daughters β and γ, and both $[\![\beta]\!]^g$ and $[\![\gamma]\!]^g$ are of type $\langle e, t\rangle$, then, for any assignment g,

$$[\![\alpha]\!]^g = \left[\lambda x_e.\, T \text{ iff } [\![\beta]\!]^g(x) = T \text{ and } [\![\gamma]\!]^g(x) = T\right].$$

Pronoun Rule (PR). If α is a pronoun or a trace bearing the index i, then, for any assignment g,

$$[\![\alpha_i]\!]^g = g(i).$$

Predicate Abstraction (PA). If α is a branching node whose daughters are β_i and γ, where β_i is a binder index i or a relative pronoun bearing the index i, then, for any assignment g,

$$[\![\alpha]\!]^g = \lambda x_e.\, [\![\gamma]\!]^{g[i \mapsto x]}$$

where $g[i \mapsto x]$ is just like g except that it maps i to x.